

LAPORAN PRAKTIKUM
STRUKTUR DATA
ALGORITMA SORTING



OLEH:
DEVINA AMANDA PUTRI
(2411533009)

DOSEN PENGAMPU:
DR. WAHYUDI, S.T, M.T

FAKULTAS TEKNOLOGI INFORMASI
DEPARTEMEN INFORMATIKA
UNIVERSITAS ANDALAS

2024

A. PENDAHULUAN

Sorting atau pengurutan data merupakan salah satu konsep penting dalam struktur data yang sering digunakan dalam pengolahan informasi. Sorting biasanya digunakan untuk menyusun data dalam urutan tertentu, bisa dalam ascending maupun descending, sehingga proses pencarian dan analisis data menjadi lebih efisien. Dalam praktiknya, terdapat berbagai macam algoritma pengurutan yang masing-masing memiliki karakteristik, kelebihan dan kekurangan tersendiri.

Pada praktikum kali ini diimplementasikan empat algoritma sorting berbeda, yaitu Bubble Sort, Shell Sort, Quick Sort dan Merge Sort dalam program Java berbasis antarmuka grafis atau GUI. Setiap algoritma memiliki mekanisme yang berbeda dalam menyusun data. Pada Bubble Sort menggunakan pendekatan pertukaran berulang antar elemen yang berdekatan, Shell Sort mengembangkan konsep insertion sort dengan jarak tertentu atau gap, Quick Sort menerapkan prinsip *divide and conquer* dengan memilih pivot sebagai pusat pengurutan, sedangkan Merge Sort membagi array menjadi bagian-bagian kecil lalu menggabungkannya kembali dalam keadaan terurut.

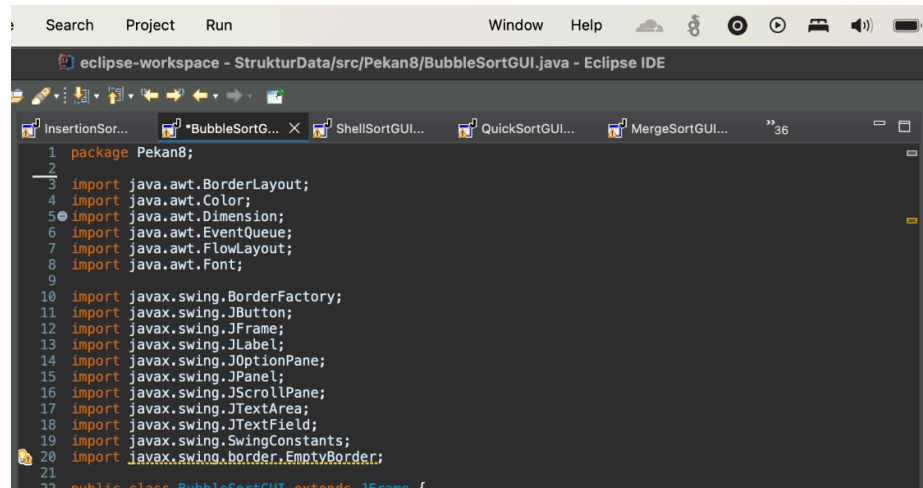
B. TUJUAN

1. Memahami algoritma Insertion Sort dan Selection Sort
2. Mengimplementasikan algoritma tersebut dalam program Java menggunakan Graphical User Interface atau GUI

C. LANGKAH-LANGKAH Pengerjaan

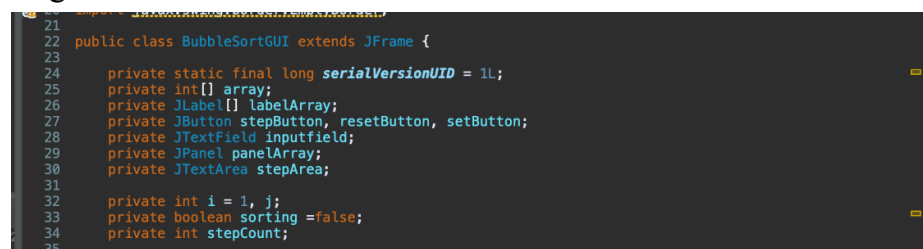
a. BubbleSortGUI.java

1. Buat class baru dengan Java GUI pada package pekan 8. Import library Java AWT dan Swing untuk kebutuhan GUI



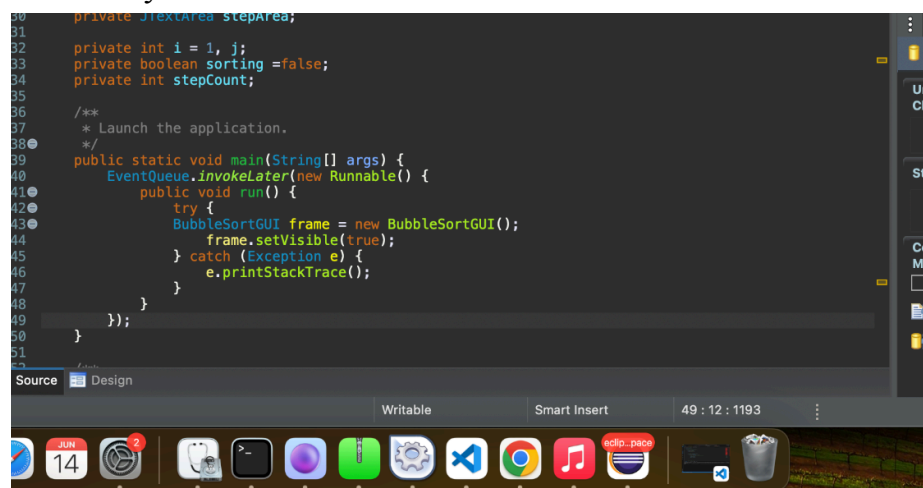
```
1 package Pekan8;
2
3 import java.awt.BorderLayout;
4 import java.awt.Color;
5 import java.awt.Dimension;
6 import java.awt.EventQueue;
7 import java.awt.FlowLayout;
8 import java.awt.Font;
9
10 import javax.swing.BorderFactory;
11 import javax.swing.JButton;
12 import javax.swing.JFrame;
13 import javax.swing.JLabel;
14 import javax.swing.JOptionPane;
15 import javax.swing.JPanel;
16 import javax.swing.JScrollPane;
17 import javax.swing.JTextArea;
18 import javax.swing.JTextField;
19 import javax.swing.SwingConstants;
20 import javax.swing.border.EmptyBorder;
```

2. Membuat class turunan dari JFrame untuk membuat jendela GUI, lalu deklarasi komponen-komponen GUI yang akan digunakan. Tambahkan juga i dan j untuk iterasi Bubble Sort, sorting untuk status sorting dan stepCount untuk mencatat langkah



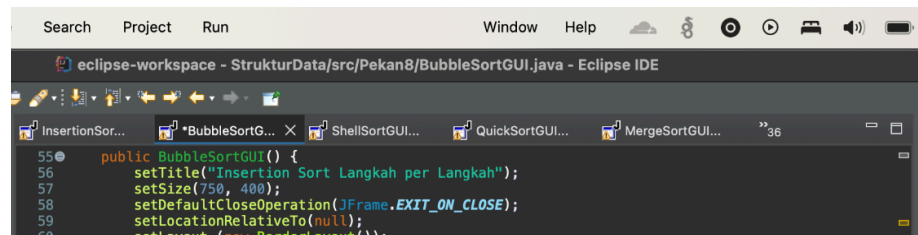
```
21
22 public class BubbleSortGUI extends JFrame {
23
24     private static final long serialVersionUID = 1L;
25     private int[] array;
26     private JLabel[] labelArray;
27     private JButton stepButton, resetButton, setButton;
28     private JTextField inputfield;
29     private JPanel panelArray;
30     private JTextArea stepArea;
31
32     private int i = 1, j;
33     private boolean sorting = false;
34     private int stepCount;
35 }
```

3. Buat fungsi utama yang dijalankan pertama kali saat program dimulai. Jalankan GUI secara aman di event dispatch thread. Lalu membuat objek GUI dari class BubbleSortGUI dan tampilkan GUI ke layar.



```
30 private JTextArea stepArea;
31
32 private int i = 1, j;
33 private boolean sorting = false;
34 private int stepCount;
35
36 /**
37  * Launch the application.
38  */
39 public static void main(String[] args) {
40     EventQueue.invokeLater(new Runnable() {
41         public void run() {
42             try {
43                 BubbleSortGUI frame = new BubbleSortGUI();
44                 frame.setVisible(true);
45             } catch (Exception e) {
46                 e.printStackTrace();
47             }
48         }
49     });
50 }
51 }
```

4. Buatlah konstruktor class yang akan mengatur dan menyiapkan semua elemen GUI. Tentukan judul jendela, ukuran dan menutup jendela jika program jendela di tutup.



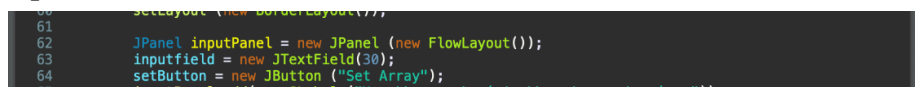
```
55 public BubbleSortGUI() {
56     setTitle("Insertion Sort Langkah per Langkah");
57     setSize(750, 400);
58     setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
59     setLocationRelativeTo(null);
60 }
```

Set layout utama jendela menjadi BorderLayout(bagian atas, tengah, bawah, kanan, kiri)



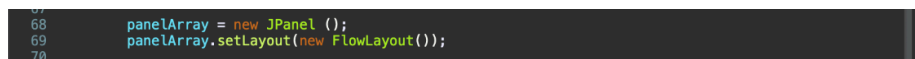
```
60     setLayout(new BorderLayout());
61 }
```

Panel input menggunakan layout horizontal dan berisi kolom input teks diberikan selebar 30 karakter lalu tombol



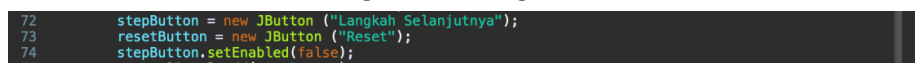
```
61 JPanel inputPanel = new JPanel(new FlowLayout());
62 inputfield = new JTextField(30);
63 setButton = new JButton("Set Array");
```

Panel untuk menampilkan array dalam bentuk tabel



```
68 panelArray = new JPanel();
69 panelArray.setLayout(new FlowLayout());
70 }
```

Tombol kontrol untuk langkah sorting dan reset



```
72 stepButton = new JButton("Langkah Selanjutnya");
73 resetButton = new JButton("Reset");
74 stepButton.setEnabled(false);
```

Panel bawah kontrol berisi dua tombol, area teks log 8 baris dan kolom 60, tidak bisa di edit oleh user dan scroll bar untuk area log langkah



```
70 JPanel controlPanel = new JPanel();
71 stepButton = new JButton("Langkah Selanjutnya");
72 resetButton = new JButton("Reset");
73 stepButton.setEnabled(false);
74 controlPanel.add(stepButton);
75 controlPanel.add(resetButton);
76
77 stepArea = new JTextArea(8, 60);
78 stepArea.setEditable(false);
79 stepArea.setFont(new Font("Monospaced", Font.PLAIN, 14));
80 JScrollPane scrollPane = new JScrollPane(stepArea);
81 }
```

Dan menambahkan semua panel ke jendela utama



```
82
83
84 add(inputPanel, BorderLayout.NORTH);
85 add(panelArray, BorderLayout.CENTER);
86 add(controlPanel, BorderLayout.SOUTH);
87 add(scrollPane, BorderLayout.EAST);
88 }
```

5. Method setArrayFromInput untuk memproses input dari field menjadi array integer. Mengambil teks dari inputfield lalu menghapus spasi di awal dan akhir. Jika input kosong, maka langsung keluar dari method tanpa memproses apa pun. Memecah input berdasarkan koma menjadi array string dan menginisialisasi array integer dengan panjang yang sama seperti jumlah elemen input.


```

93 private void setArrayFromInput() {
94     String text = inputField.getText().trim();
95     if (text.isEmpty()) return;
96     String[] parts = text.split(",");
97     array = new int[parts.length];
98 }

```

Mencoba mengonversi setiap elemen string ke integer dan menyimpan ke array. trim() dipakai agar spasi tidak bikin error

```

98     try {
99         for (int k = 0; k < parts.length; k++) {
100             array[k] = Integer.parseInt(parts[k].trim());
101         }
102     } catch (NumberFormatException e) {
103         JOptionPane.showMessageDialog(this, "Masukkan hanya angka yang dipisahkan "
104             + "dengan koma!", "Error", JOptionPane.ERROR_MESSAGE);
105         return;
106     }

```

Jika input mengandung karakter non-angka, program akan menampilkan pesan error dan keluar dari method

```

101     } catch (NumberFormatException e) {
102         JOptionPane.showMessageDialog(this, "Masukkan hanya angka yang dipisahkan "
103             + "dengan koma!", "Error", JOptionPane.ERROR_MESSAGE);
104         return;
105     }

```

Mereset indeks, langkah dan status sorting, tombol step diaktifkan dan area log dikosongkan. Menghapus elemen visual sebelumnya dan membuat array JLabel baru untuk elemen array

```

105     i = 0;
106     j = 0;
107     stepCount = 1;
108     sorting = true;
109     stepButton.setEnabled(true);
110     stepArea.setText("");
111     panelArray.removeAll();
112     labelArray = new JLabel[array.length];

```

Loop untuk membuat tampilan kotak angka atau label untuk tiap elemen array dengan gaya dan ukuran tertentu, lalu menambahkannya ke panel.

```

113     for (int k = 0; k < array.length; k++) {
114         labelArray[k] = new JLabel(String.valueOf(array[k]));
115         labelArray[k].setFont(new Font("Arial", Font.BOLD, 24));
116         labelArray[k].setOpaque(true);
117         labelArray[k].setBackground(Color.WHITE);
118         labelArray[k].setBorder(BorderFactory.createLineBorder(Color.BLACK));
119         labelArray[k].setPreferredSize(new Dimension(50, 50));
120         labelArray[k].setHorizontalAlignment(SwingConstants.CENTER);
121         panelArray.add(labelArray[k]);
122     }
123

```

Memastikan panel diperbarui dan tampilan setelah perubahan

```

123
124     panelArray.revalidate();
125     panelArray.repaint();
126 }
127

```

6. Method performStep() dijalankan setiap kali tombol Step diklik untuk satu langkah sorting. Jika sorting sudah selesai atau seluruh array telah diperiksa, hentikan proses dan tampilkan pesan "Sorting selesai".

```

128 private void performStep() {
129     if (!sorting || i >= array.length) {
130         sorting = false;
131         stepButton.setEnabled(false);
132         JOptionPane.showMessageDialog(this, "Sorting selesai!");
133         return;
134     }

```

Menghapus highlight warna sebelumnya dan mempersiapkan log untuk mencatat langkah saat ini

```

133         return;
134     }
135     resetHighlights();
136     StringBuilder stepLog = new StringBuilder();

```

Menyorot dua elemen yang sedang dibandingkan dengan warna

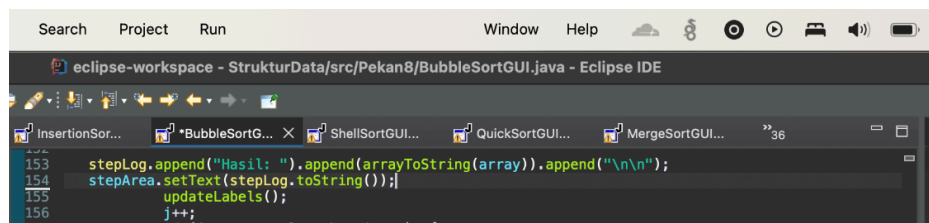
biru muda

```
135     stepLog = new StringBuilder();
136     labelArray[j].setBackground(Color.CYAN);
137     labelArray[j + 1].setBackground(Color.CYAN);
138 }
```

Jika elemen kiri lebih besar dari kanan, tukar posisi kedua elemen tersebut. Tambahkan catatan bahwa telah terjadi pertukaran ke dalam log. Jika tidak perlu ditukar, catat bahwa ada perubahan

```
138     if (array[j] > array[j + 1]) {
139         // swap
140         int temp = array[j];
141         array[j] = array[j + 1];
142         array[j + 1] = temp;
143         stepLog.append("Langkah ke-").append(stepCount)
144             .append(": Menukar elemen ke-").append(i + 1)
145             .append(" dengan ke-").append(i + 2)
146             .append(" ").append(array[i]).append(", ").append(array[i + 1]).append("\n");
147     } else {
148         stepLog.append("Langkah ke-").append(stepCount)
149             .append(": Tidak ada pertukaran antara ke-").append(i + 1)
150             .append(" dan ke-").append(i + 2).append("\n");
151     }
152 }
```

Menambahkan hasil array terbaru ke log dan menampilkannya di stepArea. Perbarui tampilan angka yang terlihat, lanjut ke indeks selanjutnya



Jika satu elemen awal sampai elemen terakhir sudah terurut, reset j, naikan i dan hitung langkah baru. Dan jika i sudah sampai ujung array, artinya sorting selesai

```
156     j++;
157     if (j == array.length - i - 1) {
158         j = 0;
159         i++;
160         stepCount++;
161         if (i >= array.length - 1) {
162             sorting = false;
163             stepButton.setEnabled(false);
164             JOptionPane.showMessageDialog(this, "Sorting selesai!");
165         }
166     }
167 }
168 }
```

7. Method `updateLabels()`, memperbarui isi label agar mencerminkan urutan angka yang baru setelah swap

```
169     private void updateLabels() {
170         for (int k = 0; k < array.length; k++) {
171             labelArray[k].setText(String.valueOf(array[k]));
172         }
173     }
174 }
```

8. Method `resetHighlights()`, menghapus warna sorotan sebelumnya dari semua elemen agar tidak menumpuk

```
175     private void resetHighlights() {
176         if (labelArray == null) return;
177         for (JLabel label : labelArray) {
178             label.setBackground(Color.WHITE);
179         }
180     }
```

9. Method `reset()`, mereset seluruh tampilan dan status aplikasi agar bisa digunakan dari awal lagi

```
182 private void reset() {
183     inputField.setText("");
184     panelArray.removeAll();
185     panelArray.revalidate();
186     panelArray.repaint();
187     stepArea.setText("");
188     stepButton.setEnabled(false);
189     sorting = false;
190     i = 0;
191     j = 0;
192     stepCount = 1;
193 }
194
```

10. Method `arrayToString(int[] arr)`, mengubah array integer ke format string yang bisa ditampilkan di log, dipisahkan oleh koma.

```
195 private String arrayToString(int[] arr) {
196     StringBuilder sb = new StringBuilder();
197     for (int k = 0; k < arr.length; k++) {
198         sb.append(arr[k]);
199         if (k < arr.length - 1) sb.append(", ");
200     }
201     return sb.toString();
202 }
203
```

b. ShellSortGUI.java

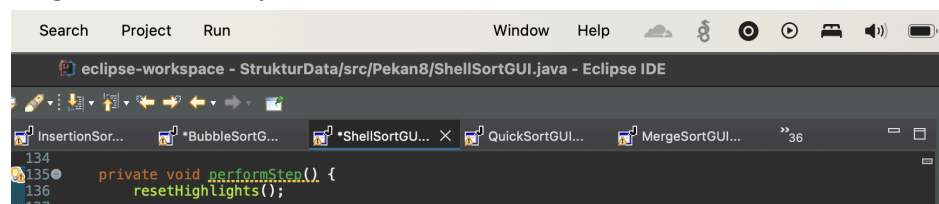
1. Class `ShellSortGUI` merupakan pengembangan dari `InsertionSortGUI` dengan perubahan pada bagian logika sorting, yaitu menggunakan algoritma Shell Sort. Struktur GUI, komponen dan fungsionalitas dasar lainnya masih sama seperti class sebelumnya
2. Deklarasi variabel baru, yaitu `gap`. Variabel ini digunakan untuk menyimpan nilai `gap` untuk iterasi Shell Sort sebagai jarak antara elemen-elemen yang akan dibandingkan dan ditukar.

```
private int gap;
```

3. Di dalam method `setArrayFromInput()`, inisialisasi `gap` dimulai dari setengah panjang array (aturan umum dalam shell sort). Nilai ini akan dikurangi setengahnya tiap selesai satu pass sorting

```
109
110     gap = array.length / 2;
111     i = gap;
```

4. Method `performStep()`, method ini akan dieksekusi setiap tombol Step ditekan dan merepresentasikan satu langkah dalam algoritma Shell Sort. `resetHighlights` untuk membersihkan warna dari langkah sebelumnya.



```
134
135 private void performStep() {
136     resetHighlights();
137 }
```

Jika proses sorting dihentikan atau `gap` sudah 0 yang artinya tidak ada jarak lagi untuk membandingkan elemen maka sorting

selesai.

```
136     resetHighlights();
137
138     if (!sorting || gap == 0) {
139         sorting = false;
140         stepButton.setEnabled(false);
141         JOptionPane.showMessageDialog(this, "Sorting selesai!");
142         return;
143     }
144
```

Mengecek apakah masih ada elemen yang belum diproses dengan i. Jika belum ada proses swap: simpan indeks i ke j, simpan elemen saat ini dengan (array[i]) ke variabel sementara temp. Tandai bahwa kita sedang dalam proses swapping.

```
144
145     if (i < array.length) {
146         if (!isSwapping) {
147             j = i;
148             temp = array[i];
149             isSwapping = true;
150         }
151
```

Cek apakah elemen di jarak gap sebelum j lebih besar dari temp. Kalau ya, berarti perlu digeser ke kanan. Geser elemen yang lebih besar ke posisi saat ini (j) dan membuat ruang bagi temp. Tandai elemen yang baru digeser dengan warna hijau dan elemen yang dibandingkan dengan warna biru

```

152         if (j >= gap && array[j - gap] > temp) {
153             array[j] = array[j - gap];
154             labelArray[j].setBackgroundColor(Color.GREEN);
155             labelArray[j - gap].setBackgroundColor(Color.CYAN);
156
```

Update tampilan GUI, catat aksi penggeseran, lalu majukan j ke kiri dengan jarak gap untuk lanjut membandingkan elemen sebelumnya

```

157         updateLabels();
158         logStep("Geser elemen " + array[j] + " ke kanan");
159         j -= gap;
160
```

Jika tidak perlu digeser lagi, tempatkan temp ke posisi j, perbarui tampilan, catat ke log bahwa temp telah ditetapkan, lanjut ke elemen berikutnya (i++) dan set bahwa proses swapping untuk elemen ini sudah selesai

```

161     } else {
162         array[j] = temp;
163         updateLabels();
164         logStep("Tempatkan " + temp + " di posisi " + j);
165         i++;
166         isSwapping = false;
167     }
168
```

Jika seluruh array sudah diproses, maka kurangi nilai gap menjadi setengah. Mulai lagi dari i=gap, tambahkan catatan ke area log bahwa gap dikurangi

```

169     } else {
170         gap /= 2;
171         i = gap;
172         isSwapping = false;
173         stepArea.append("Langkah " + stepCount++ + ": Kurangi gap menjadi " + gap + "\n\n");
174     }
175 }
176
```

c. QuickSortGUI.java

1. Class QuickSortGUI merupakan pengembangan dari InsertionSortGUI dengan perubahan pada bagian logika sorting, yaitu menggunakan algoritma Quick Sort. Struktur GUI, komponen dan fungsionalitas dasar lainnya masih sama seperti class sebelumnya
2. Tambahan properti khusus untuk Quick Sort yaitu stack, untuk menyimpan stack dari indeks low dan high yang berguna mensimulasikan pemanggilan rekursif quick sort secara manual

```

34
35     private Stack<int[]> stack = new Stack<>();
36     private int low, high, pivot;

```

3. Deklarasikan variabel tambahan, seperti: low high untuk batas kiri dan kanan subarray yang sedang diproses, pivot untuk elmen acuan saat partisi, i j untuk indeks berjalan selama partisi, partitioning untuk status apakah sedang dalam proses partisi dan stepCount untuk nomor langkah visualisasi yang sedang berlangsung

```

36     private int low, high, pivot;
37     private int i, j;
38     private boolean sorting = false;
39     private boolean partitioning = false;
40     private int stepCount;
41

```

4. Buat konstruktor untuk mengatur judul jendela dengan algoritma yang dipakai, quick sort
5. Pada method setArrayFromInput, setelah array di input masukkan rentang seluruh array ke stack sebagai pekerjaan pertama quick sort

```

117     stack.clear();
118     stack.push(new int[]{0, array.length - 1});
119     partitioning = false;

```

Proses quick sort dimulai dari awal dan belum dalam mode partisi

```

120     sorting = true;
121     partitioning = false;
122     stepCount = 1;
123     stepArea.setText("");

```

6. Pada method performStep, jika stack kosong berarti sorting selesai

```

131     if (!sorting || stack.isEmpty()) {
132         sorting = false;
133         stepButton.setEnabled(false);
134         resetHighlights();
135         stepArea.append("Quick Sort selesai!\n");
136         JOptionPane.showMessageDialog(this, "Quick Sort selesai!");
137         return;
138     }
139

```

Saat belum partisi, ambil array paling atas dari stack. Tetapkan pivot sebagai elemen paling kanan(array[high]), atur i dan j untuk memulai proses partisi lalu set partitioning = true untuk lanjut ke proses pertukaran

```

139
140     if (!partitioning) {
141         int[] range = stack.pop();
142         low = range[0];
143         high = range[1];
144         pivot = array[high];
145         i = low - 1;
146         j = low;
147         partitioning = true;
148         stepArea.append("Langkah " + stepCount++ + ": Mulai partition dari index " + low + " ke "
149             highlightPivot(high);
150         return;
151     }
152

```

Tambahkan log ke area tampilan dan highlight elemen pivot dengan warna kuning

```

partitioning = true;
stepArea.append("Langkah " + stepCount++ + ": Mulai partition dari index " + low + " ke "
highlightPivot(high);

```

Selama j masih di dalam rentang yang valid untuk partisi, highlight elemen j yang sedang dibandingkan dengan pivot

```

152
153     if (j < high) {
154         highlightCompare(j, high);
155

```

Jika elemen j lebih kecil atau sama dengan pivot, naikan i namun jika i dan j berbeda, tukar elemen tersebut untuk menempatkan elemen kecil di kiri pivot. Catat pertukaran ke log langkah

```

154         highlightCompare(j, high);
155         if (array[j] <= pivot) {
156             i++;
157             if (i != j) {
158                 swap(i, j);
159                 stepArea.append("Langkah " + stepCount++ + ": Tukar " + array[i] + " dan " + arra

```

Jika elemen lebih besar dari pivot, lewaji saja. Lanjutkan dengan j++

```

163     } else {
164         stepArea.append("Langkah " + stepCount++ + ": Lewati " + array[j] + " karena lebih be
165     }
166     j++;
167     updateLabels();
168     return;

```

Setelah selesai scanning, tempatkan pivot di posisi tengah, yaitu di antara elemen yang lebih kecil dan lebih besar

```

171     if (i + 1 != high) {
172         swap(i + 1, high);
173         stepArea.append("Langkah " + stepCount++ + ": Pindahkan pivot ke posisi tengah\n");
174     }

```

Setelah penempatan pivot, partisi selesai. Posisi pivot final adalah p

```

174     }
175     updateLabels();
176     int p = i + 1;
177     partitioning = false;
178

```

Jika masih ada subarray di kiri dan atau kanan dari pivot, tambahkan ke stack untuk diproses nanti

```

179
180     if (p - 1 > low) stack.push(new int[]{low, p - 1});
181     if (p + 1 < high) stack.push(new int[]{p + 1, high});
182 }
183

```

7. Method highlightPivot untuk mewarnai elemen pivot sebagai kuning

```

184 private void highlightPivot(int index) {
185     resetHighlights();
186     labelArray[index].setBackgroundColor(Color.YELLOW);
187 }
188

```

8. Method `highlightCompare` menunjukkan perbandingan antara elemen `j` dan pivot

```
189 private void highlightCompare(int jIndex, int pivotIndex) {  
190     resetHighlights();  
191     labelArray[jIndex].setBackground(Color.CYAN);  
192     labelArray[pivotIndex].setBackground(Color.YELLOW);  
193 }
```

d. Class `MergeSortGUI.java`

1. Class `MergeSortGUI` merupakan pengembangan dari `InsertionSortGUI` dengan perubahan pada bagian logika sorting, yaitu menggunakan algoritma Merge Sort. Struktur GUI, komponen dan fungsionalitas dasar lainnya masih sama seperti class sebelumnya
2. Deklarasi variabel tambahan khusus merge sort, yaitu `temp` untuk menyimpan array sementara yang digunakan saat proses merge dan variabel indeks untuk proses merge: `left`, `mid`, `right`, `i`, `j` untuk iterasi bagian kiri dan kanan, `k` untuk indeks `temp`. Status proses `isMerging` `true` saat proses membandingkan dan menyalin ke `temp`, `copying` `true` saat menyalin dari `temp` ke array utama. Tambahkan linkedlist khusus merge sort untuk menyimpan daftar range yang akan digabung. Gunakan queue agar proses merge terjadi urutan dari bawah ke atas, sesuai konsep merge sort rekursif

```
11 private int[] array;  
12 private int[] temp;  
13 private int left, mid, right, i, j, k;  
14 private boolean isMerging = false;  
15 private boolean copying = false;  
16  
17 private JLabel[] labelArray;  
  
22 private Queue<int[]> mergeQueue = new LinkedList<>();  
23  
24
```

3. Pada method `setArrayFromInput()`, kosongkan antrian merge yang lama lalu panggil fungsi rekursif `generateMergeStep()` untuk mengisi antrian `mergeQueue` dengan semua range yang perlu digabung

```
101         panelArray.add(labelArray[1]);  
102     }  
103     mergeQueue.clear();  
104     generateMergeStep(0, array.length - 1);  
105     stepButton.setEnabled(true);  
106     stepArea.setText("");  
107     stepCount = 1;  
108     isMerging = false;  
109     copying = false;  
110     panelArray.revalidate();  
111     panelArray.repaint();  
112 }  
113
```

4. Method `generateMergeStep()`, versi rekursif dari proses merge sort tapi hanya menyimpan rencana penggabungan, belum benar-benar menggabungkan. `mergeQueue.add` menyimpan

setiap pasangan yang akan digabung untuk diproses nanti secara bertahap di tombol langkah selanjutnya

```

195
196 private void generateMergeStep(int left, int right) {
197     if (left < right) {
198         int mid = (left + right) / 2;
199         generateMergeStep(left, mid);
200         generateMergeStep(mid + 1, right);
201         mergeQueue.add(new int[]{left, mid, right});
202     }
203 }
204 }
205

```

5. Method performStep(), jika belum dalam kondisi isMerging, ambil satu proses merge dari MergeQueue. Tentukan batas-batas dan siapkan array temp

```

114 private void performStep() {
115     resetHighlights();
116
117     if (!isMerging && !mergeQueue.isEmpty()) {
118         int[] range = mergeQueue.poll();
119         left = range[0];
120         mid = range[1];
121         right = range[2];
122         temp = new int[right - left + 1];
123         i = left;
124         j = mid + 1;
125         k = 0;
126         isMerging = true;
127         stepArea.append("Langkah " + stepCount++ + ": Bandingkan dan salin elemen \n");
128         return;
129     }
130 }

```

Bandingkan elemen kiri dan kanan, salin yang lebih kecil ke temp

```

130
131     if (isMerging) {
132         if (i <= mid && j <= right) {
133             labelArray[i].setBackground(Color.CYAN);
134             labelArray[j].setBackground(Color.CYAN);
135             if (array[i] <= array[j]) {
136                 temp[k++] = array[i++];
137             } else {
138                 temp[k++] = array[j++];
139             }
140             stepArea.append("Langkah " + stepCount++ + ": Bandingkan dan salin \n");
141             return;
142         }
143     }

```

Salin sisa elemen jika salah satu sisi habis, jika hanya bagian kiri atau kanan yang masih punya elemen langsung salin

```

143     } else if (i <= mid) {
144         temp[k++] = array[i++];
145         stepArea.append("Langkah " + stepCount++ + ": Salin sisa kiri\n");
146         return;
147     } else if (j <= right) {
148         temp[k++] = array[j++];
149         stepArea.append("Langkah " + stepCount++ + ": Salin sisa kanan\n");
150         return;
151     }

```

Salin isi temp ke array utama, setelah semua elemen masuk ke temp, masuk ke fase copying

```

151     } else {
152         copying = true;
153         k = 0;
154     }

```

Proses menyalin hasil dari temp ke array utama satu persatu

```

155     if (copying && k < temp.length) {
156         array[left + k] = temp[k];
157         labelArray[left + k].setText(String.valueOf(temp[k]));
158         labelArray[left + k].setBackground(Color.GREEN);
159         k++;
160         stepArea.append("Langkah " + stepCount++ + ": Tempelkan ke array utama \n");
161         return;
162     }

```


Setelah selesai menyalin semua isi temp, lanjut ke langkah berikutnya

```
if (copying && k == temp.length) {  
    isMerging = false;  
    copying = false;  
}
```

Jika tidak ada lagi merge yang harus dilakukan dan tidak sedang dalam proses merging, berarti sorting selesai

```
if (mergeQueue.isEmpty() && !isMerging) {  
    stepArea.append("Selesai.\n");  
    stepButton.setEnabled(false);  
    JOptionPane.showMessageDialog(this, "Merge Sort selesai!");  
}
```

D. KESIMPULAN

Pada praktikum pekan 8 ini telah dipelajari dan diimplementasikan algoritma sorting yang berbeda, yaitu Bubble Sort, Shell Sort, Quick Sort dan Merge Sort ke dalam Graphical User Interface berbasis Java Swing. Setiap algoritma memiliki karakteristik, strategi dan kompleksitas yang berbeda dalam mengurutkan data serta disajikan secara interaktif mulai dari tombol dan visualisasi